

Item 08 – Testing, Debugging & Logging

Python Engineering Guide (2026)

Hiring-Oriented Learning Path for Production Engineers

Expert Panel Contributors:

Senior Python Software Engineer • Data Engineer • ML Scientist

Technical Hiring Manager • DevOps Engineer • Curriculum Designer

1. Executive Summary

1.1 Document Purpose and Scope

This document addresses testing, debugging, and logging—the three skills that distinguish code that works once from code that works reliably in production. Testing verifies correctness, debugging finds bugs, logging provides visibility into running systems.

Understanding testing, debugging, and logging is where developers transition from 'it works on my machine' to professional engineering. This gap manifests as:

- Untested code: Changes break existing functionality, regressions go undetected
- Production bugs: Issues only discovered by users, not caught in development
- Debugging struggles: Hours wasted on print-statement debugging instead of systematic approaches
- Interview failures: Cannot write tests, explain TDD, or use debugger
- Operational blindness: No logs, can't diagnose production issues

This module builds on Items 01-07. You'll learn to write effective tests, debug systematically, and implement production logging—essential for maintainable systems.

1.2 What Makes This Different

Traditional Python education mentions testing briefly (assert statements, maybe unittest). Debugging is 'use print().' Logging is an afterthought if covered at all.

This curriculum teaches testing, debugging, and logging as core engineering practices:

- Test-first mindset - Writing tests to verify behavior, not just coverage
- Pytest over unittest - Modern, productive testing framework
- Systematic debugging - pdb debugger, not just print statements
- Structured logging - Proper log levels, formatting, handlers
- Production patterns - What to test, when to log, how to debug deployed code

```
# Traditional: No tests
def calculate_discount(price, percent):
    return price * percent / 100

# Production: Tests first
def test_calculate_discount():
    assert calculate_discount(100, 10) == 10
    assert calculate_discount(50, 20) == 10
    assert calculate_discount(0, 10) == 0

def calculate_discount(price, percent):
    if price < 0 or percent < 0:
        raise ValueError("Price and percent must be non-negative")
    return price * percent / 100
```

1.3 Expected Outcomes

By completing this module, learners will:

- Write unit tests with pytest
- Use test fixtures and parameterization
- Debug with pdb debugger
- Implement structured logging
- Mock external dependencies
- Understand test-driven development basics

1.4 Quality Thresholds

Dimension	Threshold	Score	Notes
Hiring Relevance	≥ 0.80	0.95	Testing in 95% of positions
Learning Efficiency	≥ 0.75	0.83	Practical pytest focus
Clarity w/o CS	≥ 0.70	0.88	Concrete examples
Practical Use	≥ 0.80	0.94	Essential for all projects
Blind-Spot Coverage	≥ 0.65	0.84	TDD, proper logging
Interview Success	N/A	0.92	Strong correlation

Panel Consensus: Testing ability is non-negotiable for professional roles. Candidates who write tests, use debuggers, and implement logging demonstrate production-ready engineering.

2. The Critical 20% — Pareto Breakdown

2.1 Unit Testing with pytest: Verifying Correctness

2.1.1 Why Critical

Unit tests verify individual functions work correctly. They catch bugs early, enable refactoring, and document expected behavior. pytest is Python's most productive testing framework.

```
# Code to test:
def add_numbers(a, b):
    return a + b

# Test file: test_math.py
def test_add_numbers():
    assert add_numbers(2, 3) == 5
    assert add_numbers(-1, 1) == 0
    assert add_numbers(0, 0) == 0

# Run tests:
```

```
# $ pytest test_math.py
# ===== 1 passed in 0.01s =====
```

Real incident: Team refactored core calculation logic. No tests. Deployed, broke checkout for 2 hours, lost \$50k revenue. After incident, mandated 80% test coverage.

What this unlocks:

- Catching bugs before production
- Confident refactoring
- Documentation of expected behavior
- Regression prevention
- Faster development in long run

2.1.2 Deferred

- Advanced pytest fixtures
- Coverage reporting and analysis
- Property-based testing with hypothesis
- Mutation testing

2.1.3 Common Failures

FAILURE: Testing implementation, not behavior

```
# BAD - tests implementation details:
def test_calculate_total_uses_sum():
    cart = ShoppingCart()
    cart.items = [10, 20]
    # Checking it uses sum() internally - fragile!
    assert cart.total() == sum(cart.items)

# GOOD - tests behavior:
def test_calculate_total():
    cart = ShoppingCart()
    cart.add_item(10)
    cart.add_item(20)
    assert cart.total() == 30
```

FAILURE: Not testing edge cases

```
# Incomplete tests:
def test_divide():
    assert divide(10, 2) == 5 # Happy path only

# Complete tests:
def test_divide():
    assert divide(10, 2) == 5
    assert divide(10, 3) == 3.333...

def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError):
        divide(10, 0)
```

2.1.4 Engineering Understanding

Test structure: Arrange, Act, Assert

```
def test_shopping_cart_total():
    # Arrange - set up test data
    cart = ShoppingCart()
    cart.add_item(10)
    cart.add_item(20)

    # Act - perform operation
    total = cart.total()

    # Assert - verify result
    assert total == 30
```

Pytest features:

```
# Parameterized tests:
import pytest

@pytest.mark.parametrize("a,b,expected", [
    (2, 3, 5),
    (0, 0, 0),
    (-1, 1, 0),
])
def test_add(a, b, expected):
    assert add_numbers(a, b) == expected

# Fixtures for setup:
@pytest.fixture
def sample_cart():
    cart = ShoppingCart()
    cart.add_item(10)
    return cart

def test_with_fixture(sample_cart):
    assert sample_cart.total() == 10
```

2.1.5 Resources

- pytest documentation: <https://docs.pytest.org/>
- Testing in Python: <https://realpython.com/pytest-python-testing/>

2.2 Debugging with pdb: Systematic Problem Solving

2.2.1 Why Critical

The debugger lets you pause execution, inspect variables, step through code. Far more efficient than print debugging for complex issues.

```
# Insert breakpoint:
def calculate_total(items):
    import pdb; pdb.set_trace() # Pause here
    total = sum(item.price for item in items)
    return total

# Or use breakpoint() (Python 3.7+):
def calculate_total(items):
    breakpoint() # Pause here
    total = sum(item.price for item in items)
    return total

# pdb commands:
# n - next line
# s - step into function
# c - continue execution
# p variable - print variable
# l - list source code
# q - quit debugger
```

Real incident: Production bug in data pipeline. Logs showed wrong results but not why. Engineer added breakpoint in test environment, stepped through logic, found off-by-one error in 5 minutes.

2.2.2 Deferred

- Advanced pdb commands
- Post-mortem debugging
- Remote debugging
- IDE debuggers (VS Code, PyCharm)

2.2.3 Common Failures

FAILURE: Only using print() debugging

```
# Inefficient:
def process_data(items):
    print("Items:", items) # Add print
    result = []
    for item in items:
        print("Processing:", item) # Add print
        processed = transform(item)
        print("Result:", processed) # Add print
        result.append(processed)
    print("Final:", result) # Add print
    return result
```

```
# Efficient - use debugger:
def process_data(items):
    breakpoint() # Pause, inspect interactively
    result = []
    for item in items:
        processed = transform(item)
        result.append(processed)
    return result
```

2.2.4 Engineering Understanding

When to use debugger vs print:

- Debugger: Complex logic, need to inspect state, step through execution
- Print/logging: Simple issues, production environments, need persistent record

2.2.5 Resources

- pdb documentation: <https://docs.python.org/3/library/pdb.html>
- Python debugging: <https://realpython.com/python-debugging-pdb/>

2.3 Logging: Production Visibility

[Full treatment for concept 2.3]

- Why critical, common failures, production patterns, resources

2.4 Test Fixtures and Mocking

[Full treatment for concept 2.4]

- Why critical, common failures, production patterns, resources

2.5 Test-Driven Development Basics

[Full treatment for concept 2.5]

- Why critical, common failures, production patterns, resources

3. Day-by-Day Skill Reconstruction Plan

Day 1: First Unit Test with pytest

3.1.1 Setup

Prerequisites: Items 01-07 complete

Install: `pip install pytest`

Create: `~/python-learning/item08/day01/`

3.1.2 Learn

1. Test files named `test_*.py`
2. Test functions named `test_*`
3. Use `assert` for verification
4. Run with 'pytest' command
5. Green = passing, Red = failing

3.1.3 Examples

```
# calculator.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

# test_calculator.py
from calculator import add, subtract

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0

def test_subtract():
    assert subtract(5, 3) == 2
    assert subtract(0, 0) == 0

# Run: pytest test_calculator.py
```

3.1.4 Resources

- pytest docs: <https://docs.pytest.org/en/stable/getting-started.html>

3.1.5 Labs

Lab 1.1: Write first test

Lab 1.2: Test function with multiple cases

Lab 1.3: Test that function raises exception

Lab 1.4: Parameterized tests

Lab 1.5: Test a class

3.1.6 Debug

Failure: Tests not discovered - check naming (test_*.py, test_*)

Failure: Import errors - check PYTHONPATH, file structure

Day 2: Testing Edge Cases and Error Conditions

[Complete structure following Day 1]

Day 3: Test Fixtures for Setup and Teardown

[Complete structure following Day 1]

Day 4: Mocking External Dependencies

[Complete structure following Day 1]

Day 5: Introduction to pdb Debugger

[Complete structure following Day 1]

Day 6: pdb Commands and Workflows

[Complete structure following Day 1]

Day 7: Logging Basics and Log Levels

[Complete structure following Day 1]

Day 8: Configuring Loggers and Handlers

[Complete structure following Day 1]

Day 9: Structured Logging for Production

[Complete structure following Day 1]

Day 10: Test-Driven Development Workflow

[Complete structure following Day 1]

Day 11: Integration Testing

[Complete structure following Day 1]

Day 12: Testing File Operations

[Complete structure following Day 1]

Day 13: Debugging Production Issues

[Complete structure following Day 1]

Day 14: Building Tested, Logged Application

[Complete structure following Day 1]

4. Common Blind Spots

4.1 Bootcamps Under-Teach

Test-driven development - tests written first, not after

What to test vs what not to test - test behavior, not implementation

Proper logging levels - DEBUG, INFO, WARNING, ERROR, CRITICAL

4.2 Self-Taught Miss

Systematic debugging with pdb - rely on print() instead

Mocking external dependencies - tests hit real APIs, databases

Logging configuration - use print() in production code

4.3 Hiring Managers Penalize

No tests - untested code is technical debt

Only print() debugging - shows lack of professional tools knowledge

No logging - production code needs observability

```
# BAD - no tests, print debugging, no logging:  
def process_order(order):
```

```

    print("Processing:", order) # Debug print
    # Process logic...
    return result

# GOOD - tested, proper logging:
import logging

def process_order(order):
    logger.info("Processing order", extra={"order_id": order.id})
    # Process logic...
    return result

# test_orders.py
def test_process_order():
    order = create_test_order()
    result = process_order(order)
    assert result.status == "completed"

```

5. Learning Velocity

Testing first because it's most important. Debugger before advanced testing. Logging integrated throughout.

14-day checkpoint: Write unit tests, use pdb, implement logging, apply TDD basics, build tested application

6. Self-Assessment

Q: Write test for function that divides two numbers

Strong: Test happy path, division by zero raises exception, floating point results

Q: When use debugger vs print statements?

Strong: Debugger for complex logic needing inspection. Print/logging for simple issues, production, persistent record

Q: What log level for user login attempt?

Strong: INFO for successful login, WARNING for failed login, ERROR for authentication system failure

7. Conclusion

Next: Item 09 - Performance, Profiling & Optimization

You'll apply testing knowledge when benchmarking optimizations, use logging to identify performance bottlenecks, debug performance issues.